# UE23CS320B – Capstone Project Phase – 2

## Project Progress Review #1

Project Title : A Cost-Effective, Proactive Routing System for LLMs
Project ID : PW25_RG_02
Project Guide : Ravi Goripatti
Project Team : Sourabh S Mahindrakar,Chandan R ,Sreephaneesha K,Sricharan D A

# Outline

- Abstract and scope.
- Suggestions from phase-1
- Literature survey
- Design Approach
- Design constraints, Assumptions and Dependencies
- Proposed methodology/Approach
- Architecture
- Project Progress Plan for Phase 2
- References
- Any other Information

# Abstract and Scope

## 1. Introduction: Why LLM Routing is Necessary

- **Cost Optimization:** Not every query requires the most powerful (and expensive) model. Routing sends simple queries—like summarizing a short email or basic chit-chat—to cheaper, smaller models, while reserving expensive "frontier" models (like GPT-4 or Claude 3.5 Sonnet) only for complex reasoning tasks. This can reduce operational costs significantly.

- **Inefficiency of Static Deployment:** Hard-coding a single model for an application is inefficient. If you pick the small model, you fail on complex tasks (coding, math). If you pick the large model, you burn through your budget on trivial tasks.

- **The Routing Solution:** A "Router" acts as a traffic controller. It predicts the *difficulty* of a query in real-time and routes it to the cheapest model capable of answering it correctly. This allows systems to maintain "Strong Model Quality" at nearly "Weak Model Prices."

# Design constraints

## 1. Dataset Schema

Our dataset is constructed as pairs of model responses annotated with a "Preference Label." Each data sample contains four critical fields:

- **Prompt (x)**: The original user query (e.g., from GSM8K or Chatbot Arena).
- **Strong Response (y_s)**: The answer generated by the expensive model (e.g., GPT-4).
- **Weak Response (y_w)**: The answer generated by the cheap model (e.g., Llama-3-8B).
- **Label (z)**: A binary or continuous score indicating which model won.
    - 0: Weak Model is sufficient (Quality is equal or weak model is better).
    - 1: Strong Model is required (Weak model failed).

## 2.Evaluation

**Upper Bound (Strong Model):** Routing 100% of queries to **GPT-4**.

- *Result:* Max Quality, Max Cost. (The "Gold Standard" we aim to match).

## 3. Input Layer: The User Query

- **Raw Input:** The system accepts the raw natural language string from the user (e.g., *"Write a Python script to scrape a website"*). This text is converted into a high-dimensional vector v (e.g., via OpenAI's text-embedding-3-small or a local BERT model) to capture semantic meaning.
- **Scoring Function:** The router calculates a "**Compatibility Score"** by computing the interaction between the Query Vector and the Model Vector.

### 4.Output: The Probability Score

- The router outputs a scalar probability P(win_s | q) (e.g., **0.85**).
- **Interpretation:** A score of 0.85 implies an 85% confidence that the Strong Model is required to outperform the Weak Model on this query.

## 5. Final Decision (The Action)

- **Thresholding:** This probability is compared against a tunable cost threshold (alpha) (e.g., alpha = 0.5).
- **Routing Action:**
  - **If Score < Threshold:** Route to **Weak Model** (e.g., Llama-3-8B) to save cost.
  - **If Score >= Threshold:** Route to **Strong Model** (e.g., GPT-4) to ensure quality.

## Summary Flow

- **Input:** "Explain Quantum Physics" -> *[Query Vector]*
- **Router:** Calculates Difficulty Score -> *[High Complexity Detected]*
- **Output: Route to Strong Model**.

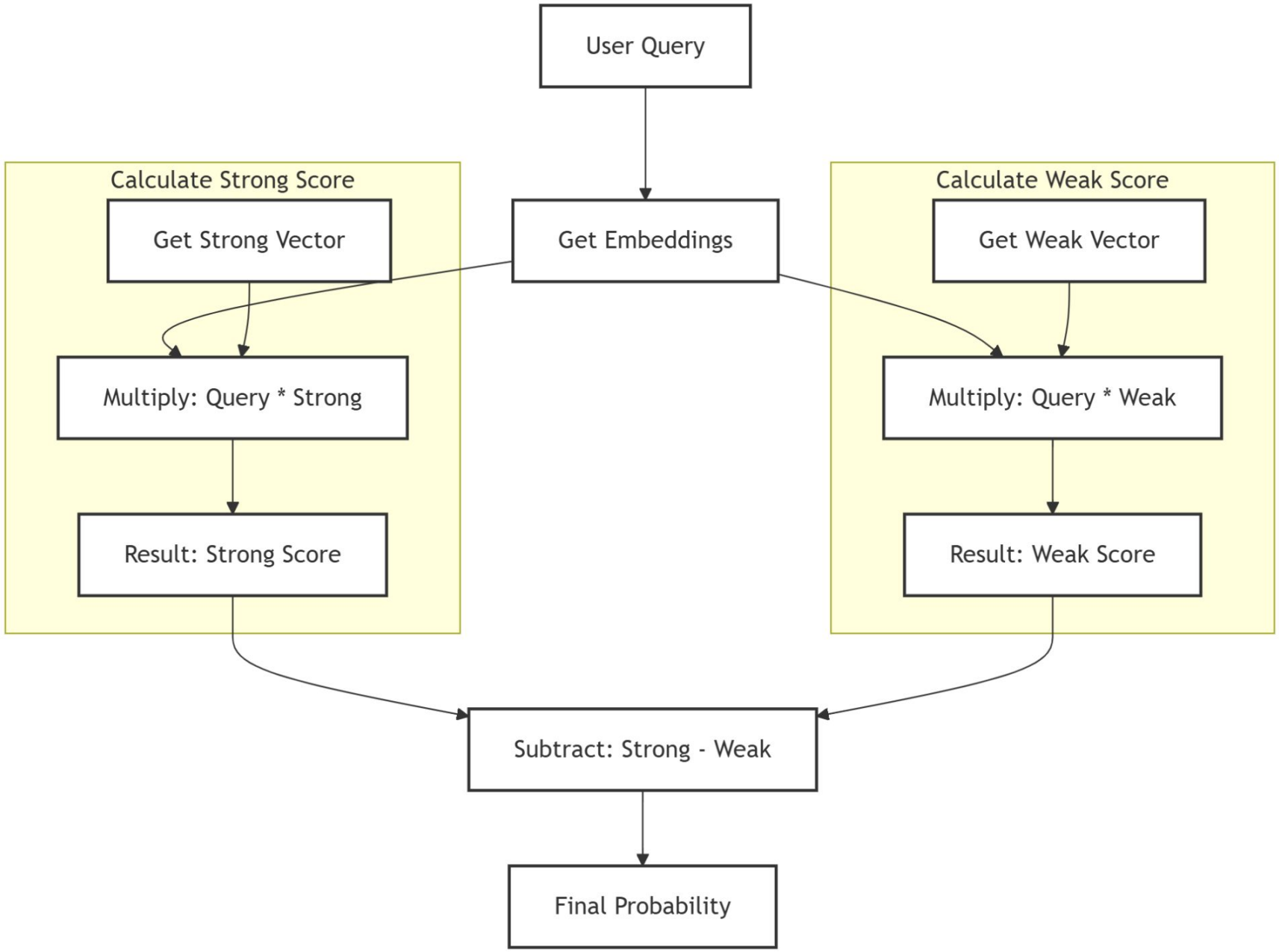
## The Matrix Factorization (MF) Technique

### The Core Intuition:

Matrix Factorization treats LLM routing like a **Netflix Recommendation System**. Instead of recommending a movie to a user, the router recommends a *Model* for a *Query*.

### How it Works:

- It assumes that "Difficulty" isn't a single number, but a relationship between the features of the query and the capabilities of the model.
- It maps every Query and every Model into a shared low-dimensional vector space (e.g., a list of 10 numbers representing attributes like "requires logic," "requires creativity," etc.).
- To predict if a model will "win" (give a good answer), it simply multiplies the Query Vector by the Model Vector (**Element Wise Multiplication**). High score = Good match.


User Query
Calculate Strong Score
Get Strong Vector
Get Embeddings
Calculate Weak Score
Get Weak Vector
Multiply: Query * Strong
Multiply: Query * Weak
Result: Strong Score
Result: Weak Score
Subtract: Strong - Weak
Final Probability

## Limitations from the literature survey:

- **Out-of-Distribution (OOD) Rigidity:** The system struggles when the data distribution changes (e.g., new task types). Retraining the router for every new task distribution is described as **"cumbersome/infeasible".(*RouteLLM)***

- **Domain-Specific Performance Failures:** ***RouteLLM*** demonstrates that routers trained on general chat data fail to generalize to reasoning tasks, achieving near-random performance on Math (GSM8K) benchmarks.

# Limitations From Architecture

## The Problem with Router (The "Averaging Trap")

- **The Limitation:** Current State-of-the-Art (**RouteLLM**) assigns a **Single Static Vector** to represent a model's capability.
- **The Consequence:** This forces a "Generalist" representation.
  - *Example:* If a Model is Excellent at **Coding** but Mediocre at **Creative Writing**, the single vector settles on an "Average" score.
  - *Result:* The router fails to route "Spiky" queries (hard math/code) correctly because the signal is diluted by the model's average performance.
- **Research Gap:** As noted in ***Hybrid LLM***, current routers lack explicit task awareness, treating specialized Logic queries indistinguishably from Chitchat.

# Proposed methodology

## Domain-Specific Matrix Factorization

### 1. The Intuition: "One Size Does Not Fit All"

- **The Baseline Problem:** Standard Matrix Factorization learns a single vector (v) for GPT-4. This forces the model to average its capabilities—it might be "**Great at Code" (+1.0)** but "**Bad at Poetry" (-1.0)**, so the vector **averages to "Okay" (0.0)**.
- **Our Solution:** Instead of one static vector, we learn **Domain-Specific Capability Vectors** for each model.
  - v_m (Vector representing GPT-4's math skill)
  - v_code (Vector representing GPT-4's coding skill)
  - v_chat (Vector representing GPT-4's chat skill)

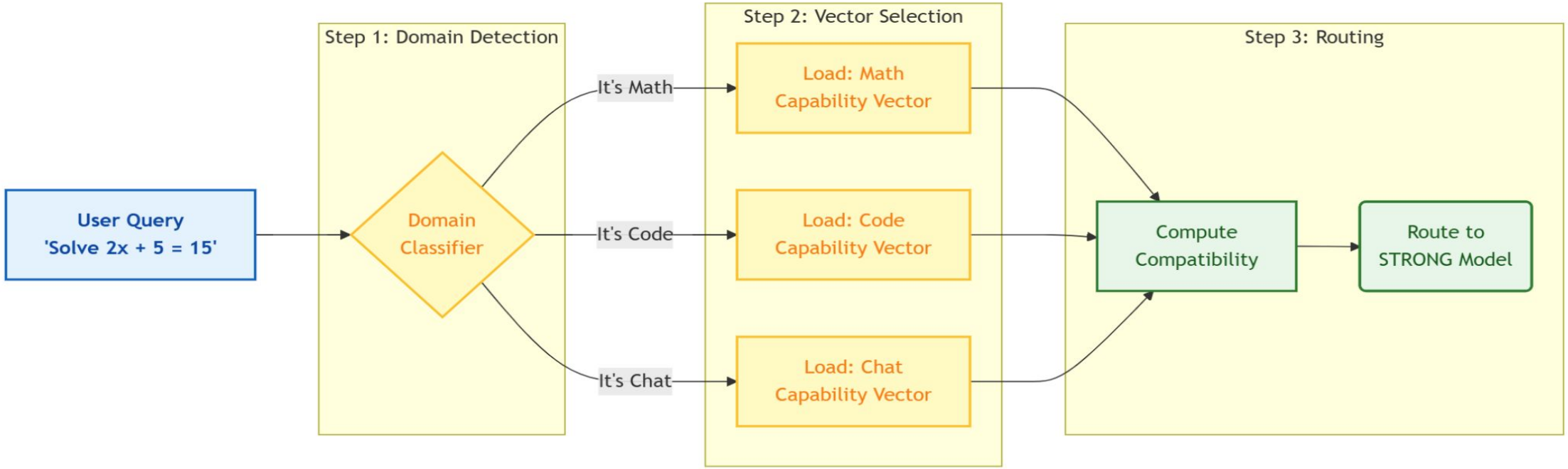
# Proposed methodology

**Mechanism:**

1. **Triage**: A **lightweight BERT classifier** (<10ms) tags the query domain (e.g., "Code").
2. **Vector Switch**: The scoring function dynamically loads the **"domain Vector"** for the Strong and Weak models.
3. **Score**: P(win) = sigma(Query.V_code).

**Why this is better:**

- **Disentangled Representation:** We stop **"Creative Writing"** data from corrupting the **"Math"** signal.


User Query
'Solve 2x + 5 = 15'
Step 1: Domain Detection
Domain Classifier
It's Math
It's Code
It's Chat
Step 2: Vector Selection
Load: Math Capability Vector
Load: Code Capability Vector
Load: Chat Capability Vector
Step 3: Routing
Compute Compatibility
Route to STRONG Model

**What We Implemented:**

1.**Matrix Factorization** approach with architecture details (2×128 learnable matrix, 384-dim query embeddings)

2.**Verification**: Training results showing the model works (validation accuracy ~64%, successful convergence)

3.**Dataset Structure**:

Model pair: GPT-4 (strong) vs Llama-2-7b (weak)

Sample structure with all fields explained (prompt, responses, labels, rankings)

**Nectar Dataset**

## Evaluation Results on Unbalanced Test Set

(62.44% GPT-4 wins, 37.56% Llama wins)

**Baseline:** Always using GPT-4 = 62.60% quality

Performance at Different Drop Targets:

| Target Drop | Actual Drop | Quality | Strong Usage | Weak Usage | Cost Advantage | Threshold |
|---|---|---|---|---|---|---|
| 0 | 0 | 62.6 | 100.00% | 0.00% | 0.00% | 0 |
| 0.5 | 0.48 | 62.12 | 73.33% | 26.67% | 26.67% | 0.423 |
| 1 | 0.96 | 61.64 | 68.40% | 31.60% | 31.60% | 0.437 |
| 1.5 | 1.38 | 61.22 | 64.69% | 35.31% | 35.31% | 0.447 |

**Key Metrics at 1% Performance Drop:**

**Router Quality:** 61.64% (vs 62.60% baseline)

**Routes** 31.6% of queries to Llama-2-7b (saving 31.6% of GPT-4 calls)

**Routes** 68.4% of queries to GPT-4

**Threshold:** 0.437

## References


- **[Hybrid LLM]** Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Rühle, V., ... & Awadallah, A. H. (2024). **Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing.** *Proceedings of the International Conference on Learning Representations (ICLR).*
- **[RouteLLM]** Ong, I., Almahairi, A., Wu, V., Chiang, W. L., Wu, T., Gonzalez, J. E., ... & Stoica, I. (2025). **RouteLLM: Learning to Route LLMs with Preference Data.** *Proceedings of the International Conference on Learning Representations (ICLR).*
- **[Nectar Dataset]** Zhu, B., Frick, E., Wu, T., Zhu, H., & Jiao, J. (2023). **Starling-7B: Improving Helpfulness and Harmlessness with RLAIF.** *arXiv preprint arXiv:2311.12711.* (The Nectar dataset was released as part of the Starling-7B project).

# Thank You